

Clase de Ejercicios

Listas, Índices y Ciclos

Clase de Programación

Profesor: Mateo Taito Stambuk

Email: mateo.taitos@edu.uai.cl

5 de Junio, 2026

Objetivos de la Clase

- **Aplicar** la metodología de resolución vista el 03/06
- **Practicar** el uso de `.index()` sobre listas paralelas
- **Imprimir** figuras en pantalla usando ciclos anidados
- **Trabajar** con listas 2D de largo irregular

Recordatorio: Metodología 03/06

Planificar → Diagrama → Implementar → Revisar

1. Planificar

- Leer el problema completo
- Identificar variables
- Definir la lógica
- Pensar las salidas
- Anotar restricciones

2. Diagrama de Flujo

- Visualizar el flujo lógico
- Decidir: ¿ciclo? ¿condición?
- Marcar entradas y salidas

3. Implementar

- Traducir el diagrama a código
- Seguir la estructura: inicialización → lógica → salidas

4. Revisar

- Probar con casos de prueba
- Verificar que el resultado sea correcto
- Si falla, volver al paso 1

1 Listas Paralelas con .index()

Ejercicio 1: Análisis de Ventas Mensuales

Enunciado del Problema

Una tienda registra las ventas de cada mes del año. Tenemos dos listas paralelas:

- Una lista con los **12 meses** del año.
- Una lista con las **ventas** correspondientes (mismo orden).

Misión: A partir de los datos, calcular y mostrar:

1. El **total** de ventas del año.
2. El **promedio mensual** de ventas.
3. El **mejor mes** (nombre + cantidad de ventas).
4. El **peor mes** (nombre + cantidad de ventas).

Ejercicio 1: Planteamiento

Planificar: Variables, Lógica, Salidas

Variables

- `meses` : lista de strings (12 elementos)
- `ventas` : lista de números (12 elementos)
- `total` : acumulador de ventas
- `promedio` : $\text{total} / 12$
- `max_ventas` , `min_ventas` : valores extremos
- `pos_mejor` , `pos_peor` : posiciones

Lógica

- Calcular `total` con `sum(ventas)`
- `promedio = total / len(ventas)`
- Usar `max(ventas)` y `min(ventas)` para extremos
- Usar `.index()` para ubicar la posición
- Conectar con `meses[pos]` para el nombre

Salidas esperadas

Total de ventas del año: 2050
Promedio mensual: 170.83 ventas
Mejor mes: Diciembre (220 ventas)
Peor mes: Enero (120 ventas)

Ejercicio 1: Diagrama de Flujo

Ejercicio 1: Código Completo

```
# 1. Inicialización: datos del problema
meses = ["Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
         "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"]
ventas = [120, 145, 160, 130, 175, 200, 210, 195, 180, 165, 150, 220]

# 2. Lógica: cálculos y búsquedas
total = sum(ventas)
promedio = total / len(ventas)
max_ventas = max(ventas)
min_ventas = min(ventas)
pos_mejor = ventas.index(max_ventas)
pos_peor = ventas.index(min_ventas)

# 3. Salidas: mostrar los resultados
print(f"Total de ventas del año: {total}")
print(f"Promedio mensual: {promedio:.2f} ventas")
print(f"Mejor mes: {meses[pos_mejor]} ({max_ventas} ventas)")
print(f"Peor mes: {meses[pos_peor]} ({min_ventas} ventas)")
```


Ejercicio 1: Explicación Detallada

- Líneas 1-3: Definimos las dos listas paralelas: `meses` (strings) y `ventas` (números). Ambas tienen 12 elementos en el mismo orden.
- Línea 6: `sum(ventas)` recorre la lista y acumula el total. Equivale a un `for` con un acumulador.
- Línea 7: `len(ventas)` devuelve 12. Dividir el total por la cantidad da el promedio mensual.
- Líneas 8-9: `max(ventas)` y `min(ventas)` devuelven el valor mayor y menor de la lista.
- Líneas 10-11: Aquí está la clave: `.index(valor)` recorre la lista y devuelve la posición donde aparece ese valor.
 - `ventas.index(220) → 11` (porque 220 está en la posición 11)
- Línea 14: Combinamos `meses[11] → "Diciembre"` con el valor `220`.
- Línea 15: Mismo proceso para el peor mes, usando `pos_peor` en lugar de `pos_mejor`.

Ejercicio 1: Revisar

Caso de prueba: seguimiento paso a paso

Datos: `ventas = [120, 145, 160, 130, 175, 200, 210, 195, 180, 165, 150, 220]`

Paso	Operación	Resultado
1	<code>total = sum(ventas)</code>	<code>2050</code>
2	<code>promedio = 2050 / 12</code>	<code>170.83</code>
3	<code>max_ventas = max(ventas)</code>	<code>220</code>
4	<code>min_ventas = min(ventas)</code>	<code>120</code>
5	<code>pos_mejor = ventas.index(220)</code>	<code>11</code>

2

Impresión de Figuras

Ejercicio 2: Marco Rectangular

Enunciado del Problema

Se pide crear un programa que dibuje un **marco rectangular** en la pantalla usando el carácter asterisco (*).

El usuario ingresa (o se definen en código) el **ancho** y el **alto**.

Si `ancho = 8` y `alto = 5`, se debe ver así:

```
* * * * * * * *  
*                   *  
*                   *  
*                   *  
*                   *  
* * * * * * * *
```

- El **borde** está hecho de asteriscos.
- El **interior** está hecho de espacios en blanco.

Ejercicio 2: Planteamiento

Planificar: Variables, Lógica, Salidas

Variables

- `ancho` : número de columnas (W)
- `alto` : número de filas (H)
- `fila` : contador del ciclo externo
- `columna` : contador del ciclo interno

Lógica

Para cada celda `(fila, columna)`, decidir qué imprimir:

- **Borde:** `fila == 0` o `fila == alto - 1` o `columna == 0` o `columna == ancho - 1` → `"* "`
- **Interior:** en cualquier otro caso → `" "` (dos espacios)

Truco visual

Ejercicio 2: Diagrama de Flujo

Ejercicio 2: Código Completo

```
# 1. Inicialización: dimensiones del marco
ancho = 8
alto = 5

# 2. Lógica: recorrer filas y columnas
for fila in range(alto):
    for columna in range(ancho):
        es_borde = (fila == 0) or (fila == alto - 1) or (columna == 0) or (columna == ancho - 1)
        if es_borde:
            print("* ", end="")
        else:
            print("  ", end="")
    # 3. Salida: terminar la fila
    print()
```

Ejercicio 2: Explicación Detallada

- Líneas 1-2: Definimos las dimensiones del rectángulo: 8 columnas de ancho y 5 filas de alto.
- Líneas 5-6: El ciclo externo recorre las filas (de 0 a 4). El ciclo interno recorre las columnas (de 0 a 7).
- Línea 7: Calculamos `es_borde` con una condición compuesta por cuatro chequeos:
 - `fila == 0` → primera fila
 - `fila == alto - 1` → última fila
 - `columna == 0` → primera columna
 - `columna == ancho - 1` → última columna
- Líneas 8-11: Si es borde, imprimimos `"* "` (asterisco + espacio). Si no, imprimimos `" "` (dos espacios) para mantener el alineamiento.
- Línea 13: Después de imprimir todos los caracteres de una fila, `print()` sin argumentos genera el salto de línea.

Ejercicio 2: Revisar

Caso de prueba: ancho = 4, alto = 3

Seguimiento de variables

Iter	fila	col	¿Borde?	Imprime
1	0	0	Sí	*
2	0	1	Sí	*
3	0	2	Sí	*
4	0	3	Sí	*
5	1	0	Sí	*
6	1	1	No	

Salida esperada

```
* * * *  
*     *  
* * * *
```

Funciona para cualquier tamaño de marco

3

Listas 2D Irregulares

Ejercicio 3: Promedio con Evaluaciones Variables

Enunciado del Problema

Un profesor registra las notas de sus estudiantes, pero cada estudiante tiene una cantidad distinta de evaluaciones (algunos dieron 3 pruebas, otros 5, etc.).

```
nombres = ["Ana", "Juan", "Pedro", "Sofía", "Luis"]
notas = [
    [5.5, 6.0, 4.8],          # Ana: 3 notas
    [6.2, 5.8, 6.5, 7.0, 5.5], # Juan: 5 notas
    [4.0, 5.5],              # Pedro: 2 notas
    [6.0, 5.5, 6.8, 6.2],    # Sofía: 4 notas
    [5.0]                    # Luis: 1 nota
]
```

Misión: Calcular y mostrar:

1. El promedio de cada estudiante (mostrando su nombre)
2. El estudiante con mejor promedio (nombre + promedio)
3. El promedio general del curso (todas las notas combinadas)

Ejercicio 3: Planteamiento

Planificar: Variables, Lógica, Salidas

Variables

- `nombres` : lista de strings (5 elementos)
- `notas` : lista 2D irregular (listas internas de largo variable)
- `promedios` : lista 1D con el promedio de cada estudiante
- `suma_total` , `cantidad_total` : para el promedio del curso

Lógica (clave del ejercicio)

Como cada lista interna tiene un largo distinto, no podemos asumir un número fijo de columnas.

- Usamos `len(notas[i])` para preguntar cuántas evaluaciones tiene ese estudiante
- El ciclo interno va hasta `len(notas[i])` , no hasta un número fijo

Diferencia con matrices "normales"

En una matriz regular (ej: 3×3), todas las listas internas tienen largo 3. Aquí cada `notas[i]` puede tener cualquier largo entre 1 y N.

Ejercicio 3: Diagrama de Flujo

Ejercicio 3: Código Completo

```
# 1. Inicializacion (datos ya mostrados en enunciado)
promedios = []
suma_total = 0
cantidad_total = 0

# 2. Logica: recorrer listas irregulares
for i in range(len(notas)):
    suma_est = 0
    for j in range(len(notas[i])):
        suma_est += notas[i][j]
    promedio = suma_est / len(notas[i])
    promedios.append(promedio)
    suma_total += suma_est
    cantidad_total += len(notas[i])

# 3. Salidas: mostrar resultados
for i in range(len(nombres)):
    print(f"{nombres[i]}: promedio {promedios[i]:.2f}")

pos_mejor = promedios.index(max(promedios))
print(f"Mejor: {nombres[pos_mejor]} con {promedios[pos_mejor]:.2f}")
print(f"Promedio del curso: {suma_total / cantidad_total:.2f}")
```

Ejercicio 3: Explicación Detallada

- Líneas 1-9: Definimos los datos. Observa cómo las listas internas tienen largos diferentes: 3, 5, 2, 4, 1.
- Líneas 11-13: Inicializamos `promedios` (lista vacía) y los acumuladores para el promedio del curso.
- Línea 16: Clave del ejercicio: `range(len(notas))` itera por la cantidad de estudiantes (5).
- Línea 19: La diferencia fundamental: `range(len(notas[i]))` itera por la cantidad real de notas de ese estudiante, no por un número fijo.
 - Para Ana: `range(3)` (sus 3 notas)
 - Para Juan: `range(5)` (sus 5 notas)
 - Para Pedro: `range(2)` (sus 2 notas)
- Líneas 20-21: Acumulamos y calculamos el promedio usando `len(notas[i])` como divisor.
- Líneas 25-26: Aprovechamos el ciclo para ir acumulando el total y la cantidad general del curso.
- Línea 29: Mostramos los promedios por estudiante.
- Línea 32: `promedios.index(max(promedios))` encuentra al mejor.

Ejercicio 3: Revisar

Caso de prueba: seguimiento paso a paso

Datos: 5 estudiantes con 3, 5, 2, 4, 1 notas respectivamente.

Promedio por estudiante

Estudiante	<code>len(notas[i])</code>	Suma	Promedio
Ana	3	16.3	5.43
Juan	5	31.0	6.20
Pedro	2	9.5	4.75
Sofía	4	24.5	6.13
Luis	1	5.0	5.00

Acumuladores del curso

- `suma_total = 86.3`
- `cantidad_total = 15` (3+5+2+4+1)
- `promedio_curso = 86.3 / 15 = 5.75`

Búsqueda del mejor

- `max(promedios) = 6.20` (Juan)
- `pos_mejor = promedios.index(6.20) = 1`
- `nombres[1] = "Juan"`

Ejercicio 3: Bonus

¿Y si lo resuelvo con una lista 1D?

Paso 1: promedios a una lista 1D

```
promedios = []
for i in range(len(notas)):
    promedios.append(
        sum(notas[i]) / len(notas[i])
    )
```

Paso 2: encontrar al mejor

```
pos_mejor = promedios.index(max(promedios))
print(f"Mejor: {nombres[pos_mejor]}")
```

Dato clave: Aunque la entrada sea 2D irregular, podemos transformarla a una lista 1D de promedios y operar con herramientas 1D como `.index()` y `max()`.

Resumen de la Clase

Tres ejercicios, una metodología

P1 · Listas Paralelas

- Recorrido con `range(len( ... ))` e índice `i`
- `max()`, `min()`, `sum()` para agregaciones
- `.index(valor)` para conectar con la lista paralela
- Caso: análisis de ventas mensuales

P2 · Impresión de Figuras

- Ciclos `for` anidados (filas + columnas)
- `print( ... , end="")` para no saltar de línea
- `print()` vacío para terminar la fila
- Condiciones compuestas con `or`
- Caso: marco rectangular

P3 · Listas Irregulares

- Lista 2D con largos internos variables
- `len(notas[i])` como divisor y tope del ciclo interno
- Acumulación paralela (suma del curso)
- Caso: promedios con evaluaciones variables

Conceptos Reforzados

Lo que practicamos hoy

- Planificación antes de código: cada ejercicio pasó por Variables → Lógica → Salidas → Restricciones
- Diagramas de flujo: visualización del flujo lógico antes de implementar
- Estructura del código: inicialización → lógica → salidas
- Revisión con casos de prueba: tablas de seguimiento de variables
- Ciclo completo: planificar → implementar → revisar

Regla de oro: Un buen programador piensa más de lo que escribe, incluso en ejercicios pequeños.



Ejercicio Bonus

Torneo de un Juego

Ejercicio Bonus: Torneo de un Juego

El Desafío Final

Disponemos de los datos de un torneo donde cada jugador participó en una cantidad distinta de partidas:

```
jugadores = ["Ana", "Juan", "Pedro", "Sofía", "Luis"]
puntos_por_partida = [
    [3, 1, 2, 0, 4],      # Ana: 5 partidas
    [2, 2, 1],           # Juan: 3 partidas
    [1, 0, 1, 2],        # Pedro: 4 partidas
    [0, 1],              # Sofía: 2 partidas
    [2, 3, 1, 2, 1, 0]   # Luis: 6 partidas
]
```

Misión: A partir de los datos, calcular y mostrar:

1. Los puntos totales de cada jugador (suma de su lista interna).
2. El jugador campeón (más puntos totales) — mostrar nombre y puntos.
3. El jugador más efectivo (mejor promedio de puntos por partida) — mostrar nombre y promedio.
4. El promedio general de puntos del torneo.

Ejercicio Bonus: Pistas

Cómo abordarlo paso a paso

1. Recorrer con índice

- `range(len(puntos_por_partida))` para iterar por jugadores
- `sum(puntos_por_partida[i])` para sumar su lista interna

2. Guardar resultados parciales

- `puntos_totales = []` y `promedios = []`
- En cada iteración, `append` del cálculo

3. Conectar con la lista 1D

- `pos = puntos_totales.index(max(puntos_totales))`
- `jugadores[pos]` → nombre del campeón

4. Promedio del torneo

- Acumular puntos totales y partidas totales
- Dividir: `total_puntos / total_partidas`

Este ejercicio combina los 3 temas de la clase: listas paralelas 1D (jugadores), `.index()` (encontrar al campeón) y listas 2D irregulares (puntos por partida). ¡Intenta resolverlo solo!

¡Gracias!
¿Preguntas?

Clase de Ejercicios: Listas, Índices y Ciclos